

Use Case 1: Drilling Report Analysis Agent

Segment: Upstream | Claude Code + Bedrock AgentCore Hackathon · Energy Symposium

The Problem

Drilling engineers generate hundreds of Daily Drilling Reports (DDRs) per well — dense, unstructured text covering bit performance, mud weight, NPT events, formation tops, and operational decisions. Across a portfolio of 50+ wells, that's tens of thousands of pages of institutional knowledge sitting in PDFs, never systematically mined.

Critical patterns get buried: recurring stuck pipe causes, formation-specific bit wear signatures, mud weight windows that triggered kicks. Engineers spend hours manually searching for precedents. Costly mistakes repeat across wells and basins.

Non-productive time (NPT) accounts for 15–25% of total well costs in complex drilling programs. A single stuck pipe event averages \$500K–\$2M. If historical DDRs could surface warning signs 12–24 hours earlier, that cost is preventable. The problem isn't a lack of data — it's a lack of a system that can reason across thousands of pages of unstructured text and answer the questions engineers actually ask.

Your Task

Solve the problem above using Agentic AI — built with Claude Code and deployed on Amazon Bedrock AgentCore.

Use the dataset and reference documents provided in this package:

- **Corpus A — 75 narrative daily drilling reports** (`data/daily_drilling_reports/*.json` , 5 Permian wells)
- **Corpus B — tabular DDRs** (`data/daily_drilling_reports.csv` , 192 rows, 9 wells across 3 basins)
- **Operational context** — `npt_incident_log.csv` , `bit_records.csv` , `mud_log_summary.csv` , `formation_tops.csv` , `drilling_plan.csv` , `well_master.csv` , `well_metadata.csv` , `npt_categories.csv`
- **Benchmarks and thresholds** — `offset_well_performance.csv` , `drilling_benchmarks.csv` , `anomaly_thresholds.csv`

- **Reference documents** ([data/reference_docs/](#) , [.md](#) and [.pdf](#)) — BOP procedures (API 53), casing design reference, mud pump specifications, MWD/LWD tool reference

What you decide

Everything else. What the application actually does, which question it answers and who it answers it for, how many agents and how they divide the work, which orchestration pattern, what each tool does, which AgentCore modules you use, and what the interface looks like — all of that is your team’s call.

There is deliberately no worked solution and no capability checklist in this document. Deciding what is worth building from the problem and the data is the hackathon. Two teams solving this well should end up with two visibly different applications.

What “grounded” means here

Every number and claim your application states should trace back to a specific file, row, or passage in the data above — and it should be able to say which one. An answer that sounds expert but cites nothing is worth less than a narrower answer that shows its evidence. If the data doesn’t support a conclusion, the right behavior is to say so, not to fill the gap.

Judges score the working demo, the depth of the application’s reasoning, and whether its claims trace back to this data.

Dataset Provided

Read this first — there are two separate DDR corpora. Corpus A is narrative JSON for 5 Permian wells (keyed on [well_name](#)). Corpus B is a tabular CSV for 9 wells across 3 basins (keyed on [well_id](#)). **They use different well identifiers and do not cross-join** — a join between them returns zero rows. Pick one as your primary corpus, or handle both explicitly. Corpus A is deeper per report; Corpus B is broader across wells and basins.

Corpus A — Narrative DDRs (5 wells, Permian)

1. Daily Drilling Reports — [daily_drilling_reports/](#) — 75 JSON files (15 per well), one per rig-day

Wells [W-001](#) – [W-005](#) (“Midland State A 1H”, “Reagan County C 3H”, etc.), Midland Basin, 3 formations (Wolfcamp A, Spraberry, Bone Spring). Report dates 2024-01-15 → 2024-11-15. Filenames: [W-00N_DDR_YYYY-MM-DD.json](#) .

Each report is a flat JSON object with 62 fields covering all standard IADC sections:

Section	Key Fields
Header	report_date, well_name, api_number, operator, contractor, rig_name, field, county, state, spud_date
Depth & Progress	depth_start_ft, depth_end_ft, measured_depth_ft, true_vertical_depth_ft, footage_drilled_ft
Time Breakdown	rotating_hrs, sliding_hrs, connection_hrs, npt_hrs, npt_cause_code, npt_description, woc_hrs
Drilling Parameters	wob_klbs, rpm, torque_kftlbs, flow_rate_gpm, standpipe_pressure_psi, rop_fthr, ecd_ppg, hookload_klbs
Mud Report	mud_type, mud_weight_in_ppg, mud_weight_out_ppg, viscosity_cp, yield_point, gel_10s_10m, ph, chlorides_ppm, solids_pct
Bit Record	bit_size_in, bit_type, bit_serial, footage_on_bit_ft, hours_on_bit, dull_grade_iadc, reason_pulled
BHA	drill_collar_size_in, stabilizer_od_in, jar_type, mwd_lwd_tools
Survey	inclination_deg, azimuth_deg, dogleg_severity_deg_100ft
Casing	casing_string, casing_od_in, set_depth_ft, cement_top_ft, pressure_test_psi
Costs	daily_cost_usd, cumulative_cost_usd, afe_budget_usd
Narrative	operations_summary (free text, ~35–50 words per report)

2. Well Metadata CSV — `well_metadata.csv` — 5 rows (one per well). `well_id`, `well_name`, `api_number`, `operator`, `field`, `county`, `state`, `spud_date`, `rig_release_date`, `total_depth_ft`, `tvd_ft`, `lateral_length_ft`, `completion_type`, `target_formation`, `afe_usd`

3. NPT Incident Log CSV — `npt_incident_log.csv` — 23 incidents. `incident_id`, `well_name`, `date`, `depth_ft`, `formation`, `npt_category` (stuck_pipe / lost_circulation / well_control / equipment_failure), `root_cause`, `resolution`, `hours_lost`, `cost_usd`

4. Bit Record CSV — `bit_records.csv` — 41 bit runs. `well_name`, `run_number`, `bit_size_in`, `bit_type`, `manufacturer`, `serial_number`, `depth_in_ft`, `depth_out_ft`, `footage_ft`, `hours`, `avg_rop_fthr`, `dull_grade_iadc`, `reason_pulled`, `cost_usd`

5. Mud Log Summary CSV — `mud_log_summary.csv` — 186 depth intervals. `well_name`, `depth_ft`, `formation`, `lithology`, `c1_pct`, `c2_pct`, `c3_pct`, `ic4_pct`, `nc4_pct`, `c5_pct`, `total_gas_units`, `rop_fthr`, `show_description`

Items 3–5 join to Corpus A on `well_name` (e.g. `Midland State A 1H`), which also appears inside each JSON DDR.

Corpus B — Tabular DDRs (9 wells, 3 basins)

6. Daily Drilling Reports CSV — `daily_drilling_reports.csv` — 192 rows × 48 columns, one row per rig-day. Wells `PERM-*`, `EF-*`, `BAK-*`, `DL-*` across the Permian, Eagle Ford, and Williston basins. Report dates 2025-02-15 → 2025-02-28. Coverage is uneven by design — 3 to 39 reports per well.

Key columns: `report_id`, `well_id`, `report_date`, `report_time`, `measured_depth_ft`, `tvd_ft`, `hole_size_in`, `rop_ft_hr`, `wob_klbs`, `rpm`, `torque_ft_lbs`, `spp_psi`, `mud_weight_ppg`, `flow_rate_gpm`, `pit_volume_bbl`, `gas_units`, `activity_code`, `activity_description`, `formation`, `npt_hours`, `npt_category`, `remarks` (free text, ~12–24 words), full mud properties (`pv_cp`, `yp_lb_100sqft`, `gel_10s`, `gel_10min`, `fluid_loss_cc_30min`, `mud_chlorides_ppm`, `ph`, `mbt`, `solids_pct`), directional survey (`inclination_deg`, `azimuth_deg`, `dogleg_severity_deg_100ft`), and bit/BHA fields.

7. Well Master CSV — `well_master.csv` — 12 wells. `well_id`, `well_name`, `api_number`, `operator`, `field`, `basin`, `county`, `state`, `spud_date`, `target_formation`, `target_tvd_ft`, `planned_td_ft`, `rig_name`, `well_type`, `latitude`, `longitude`. Note: only 9 of these 12 wells have DDRs — the other 3 are planned wells with no reports yet.

8. Formation Tops CSV — `formation_tops.csv` — 39 rows across the 9 DDR wells. `well_id`, `formation`, `top_md_ft`, `top_tvd_ft`, `base_md_ft`, `base_tvd_ft`, `lithology`, `expected_mud_weight_ppg`, plus expected drilling parameters.

9. Drilling Plan CSV — `drilling_plan.csv` — 12 planned intervals for 4 wells. `well_id`, `interval_name`, planned depth/TVD ranges, and planned parameters — the baseline for planned-vs-actual analysis.

Items 7–9 join to Corpus B on `well_id` (e.g. `PERM-2847`).

Shared lookup tables (not keyed to a well)

10. Anomaly Thresholds CSV — [anomaly_thresholds.csv](#) — 49 rows. [parameter](#), [formation](#), [low_warning](#), [low_critical](#), [high_warning](#), [high_critical](#) and more. Per-formation operating envelopes.

11. NPT Categories CSV — [npt_categories.csv](#) — 18 rows. [npt_code](#), [npt_category](#), [npt_subcategory](#), [description](#), [avg_duration_hours](#), [frequency_per_well](#). The code dictionary behind [npt_category](#) / [npt_cause_code](#).

12. Drilling Benchmarks CSV — [drilling_benchmarks.csv](#) — 10 rows. Per basin/formation/hole-size benchmark ROP, WOB, RPM and more.

13. Offset Well Performance CSV — [offset_well_performance.csv](#) — 50 rows for 14 offset wells ([OFF-001](#) – [OFF-014](#)) across 8 formations. Depth-interval performance by formation. Keyed on [formation](#), so it joins to either corpus.

14. Report Templates CSV — [report_templates.csv](#) — 6 templates. [template_id](#), [template_name](#), [audience](#), [format](#), [sections](#), [include_charts](#). The report layouts this operator uses.

15. Distribution List CSV — [distribution_list.csv](#) — 6 recipients. [recipient_id](#), [name](#), [role](#), [email](#), [report_templates](#), [wells_of_interest](#). Who each report goes to.

Reference documents

16. Equipment Reference Docs — [reference_docs/](#) — 4 documents, each supplied as both [.md](#) and [.pdf](#):

- [bop_procedures](#) — BOP test procedures (API 53)
- [mud_pump_specifications](#) — mud pump specs (National / Gardner Denver)
- [mwd_lwd_tool_reference](#) — MWD/LWD tool operating ranges
- [casing_design_reference](#) — casing design reference

The Markdown versions are easier to chunk and index; the PDFs are there if you want to demo document parsing.

Logging in, environment setup, and how to submit: see the hackathon portal.

Claude Code: <https://docs.claude.com/en/docs/claude-code> **Strands Agents SDK:** <https://strandsagents.com> **Amazon Bedrock AgentCore:** <https://docs.aws.amazon.com/bedrock-agentcore/>